트랜잭션 트레이싱 시스템 분석

개념적 배경

왜 트레이싱이 필요한가?

트랜잭션 트레이싱은 Ethereum 트랜잭션의 실행 과정을 단계별로 추적하고 기록하는 시스템입니다. 이는 다음과 같은 목적으로 사용됩니다:

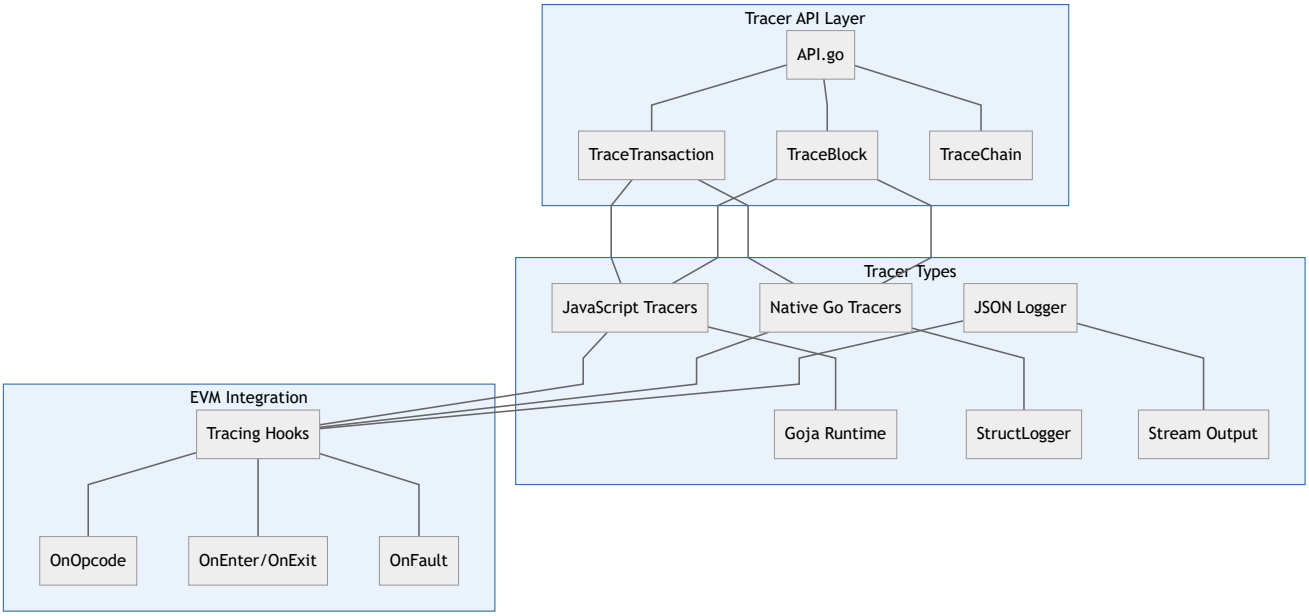
1. **디버깅**: 스마트 컨트랙트 실행 중 발생하는 문제 진단
2. **가스 최적화**: 각 opcode별 가스 소모량 분석을 통한 최적화
3. **보안 감사**: 트랜잭션 실행 흐름 분석을 통한 취약점 발견
4. **성능 분석**: EVM 실행 성능 병목 지점 식별
5. **교육 목적**: EVM 동작 원리 이해를 위한 학습 도구

다른 접근법과의 비교

- **로그 기반 추적**: 단순하지만 세부 정보 부족
- **디버거 기반**: 실시간 디버깅 가능하지만 성능 오버헤드 큼
- **정적 분석**: 빠르지만 동적 실행 정보 부족
- **geth 트레이싱**: 완전한 실행 추적과 유연한 커스터마이징 제공

구현 분석

핵심 아키텍처



주요 데이터 구조

TraceConfig 구조체

```
type TraceConfig struct {
*logger.Config
Tracer *string  // 트레이서 이름 또는 JS 코드
Timeout *string  // 실행 타임아웃
Reexec *uint64  // 재실행할 블록 수
TracerConfig json.RawMessage // 트레이서별 설정
```

```
}
```

StructLog 구조체

```
type StructLog struct {
Pc  uint64  // 프로그램 카운터
Op vm.OpCode // 실행된 opcode
Gas  uint64  // 남은 가스
GasCost  uint64  // opcode 실행 비용
Memory []byte  // 메모리 상태
MemorySize  int  // 메모리 크기
Stack []uint256.Int // 스택 상태
ReturnData []byte  // 반환 데이터
Storage  map[common.Hash]common.Hash // 스토리지 변경사항
Depth  int  // 호출 깊이
RefundCounter  uint64  // 가스 환불 카운터
Err  error  // 실행 오류
}
```

트레이서 타입별 구현

1. JavaScript 트레이서 (Goja 기반)

핵심 특징:

- 동적 트레이싱 로직 작성 가능

- 높은 유연성과 커스터마이징
- 성능 오버헤드 존재

구현 세부사항:

```
type jsTracer struct {
vm *goja.Runtime // JavaScript 런타임
env *tracing.VMContext // EVM 컨텍스트
chainConfig *params.ChainConfig // 체인 설정
toBig toBigFn // BigInt 변환 함수
toBuf toBufFn // Buffer 변환 함수
fromBuf fromBufFn // Buffer 파싱 함수
ctx  map[string]goja.Value // 컨텍스트 데이터
activePrecompiles []common.Address // 활성 프리컴파일
traceStep  bool  // step() 메서드 존재 여부
traceFrame  bool  // enter()/exit() 메서드 존재 여부
// JavaScript 함수들
result goja.Callable // result() 함수
fault goja.Callable // fault() 함수
step goja.Callable // step() 함수 (선택적)
enter goja.Callable // enter() 함수 (선택적)
exit goja.Callable // exit() 함수 (선택적)
}
```

JavaScript 트레이서 인터페이스:

```
{
// 필수 메서드
result: function(ctx, db) { /* 최종 결과 반환 */ },
fault: function(log, db) { /* 오류 처리 */ },
// 선택적 메서드
step: function(log, db) { /* 각 opcode 실행 시 호출 */ },
enter: function(frame) { /* 함수 호출 시작 시 호출 */ },
exit: function(frameResult) { /* 함수 호출 종료 시 호출 */ },
setup: function(cfg) { /* 초기 설정 */ }
}
```

2. Native Go 트레이서 (StructLogger)

핵심 특징:

- 높은 성능과 낮은 오버헤드
- 구조화된 로그 출력
- 스트리밍 및 배치 모드 지원

구현 세부사항:

```
type StructLogger struct {
cfg Config // 트레이싱 설정
env *tracing.VMContext // EVM 환경
storage  map[common.Address]Storage // 스토리지 변경 추적
output []byte  // 실행 결과
```

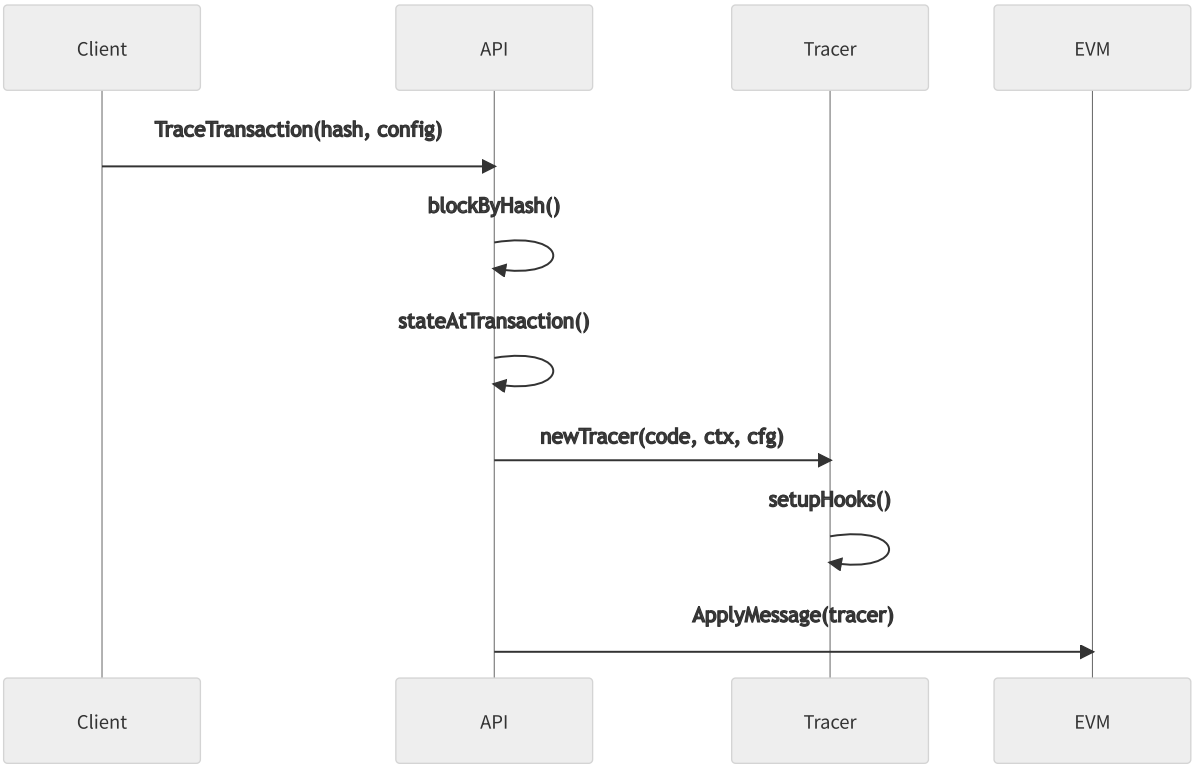
```
err error // 실행 오류
usedGas uint64 // 사용된 가스
writer io.Writer // 스트리밍 출력 (선택적)
logs []json.RawMessage // 로그 버퍼
resultSize int // 결과 크기
interrupt atomic.Bool // 중단 플래그
reason error // 중단 이유
skip bool // 시스템 콜 중 스킵
}
```

설정 옵션:

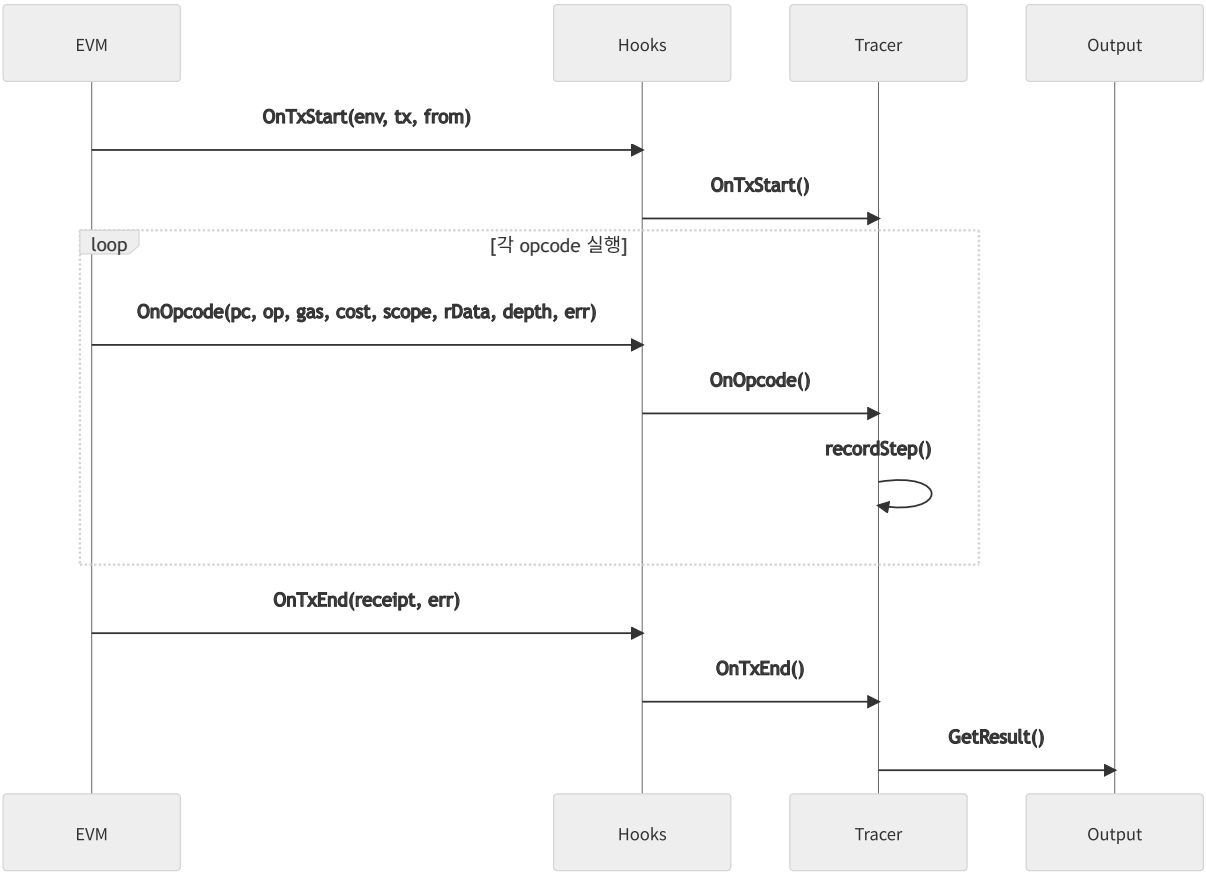
```
type Config struct {
EnableMemory bool // 메모리 캡처 활성화
DisableStack bool // 스택 캡처 비활성화
DisableStorage bool // 스토리지 캡처 비활성화
EnableReturnData bool // 반환 데이터 캡처 활성화
Limit int // 출력 크기 제한
Overrides *params.ChainConfig // 체인 설정 오버라이드
}
```

실행 흐름

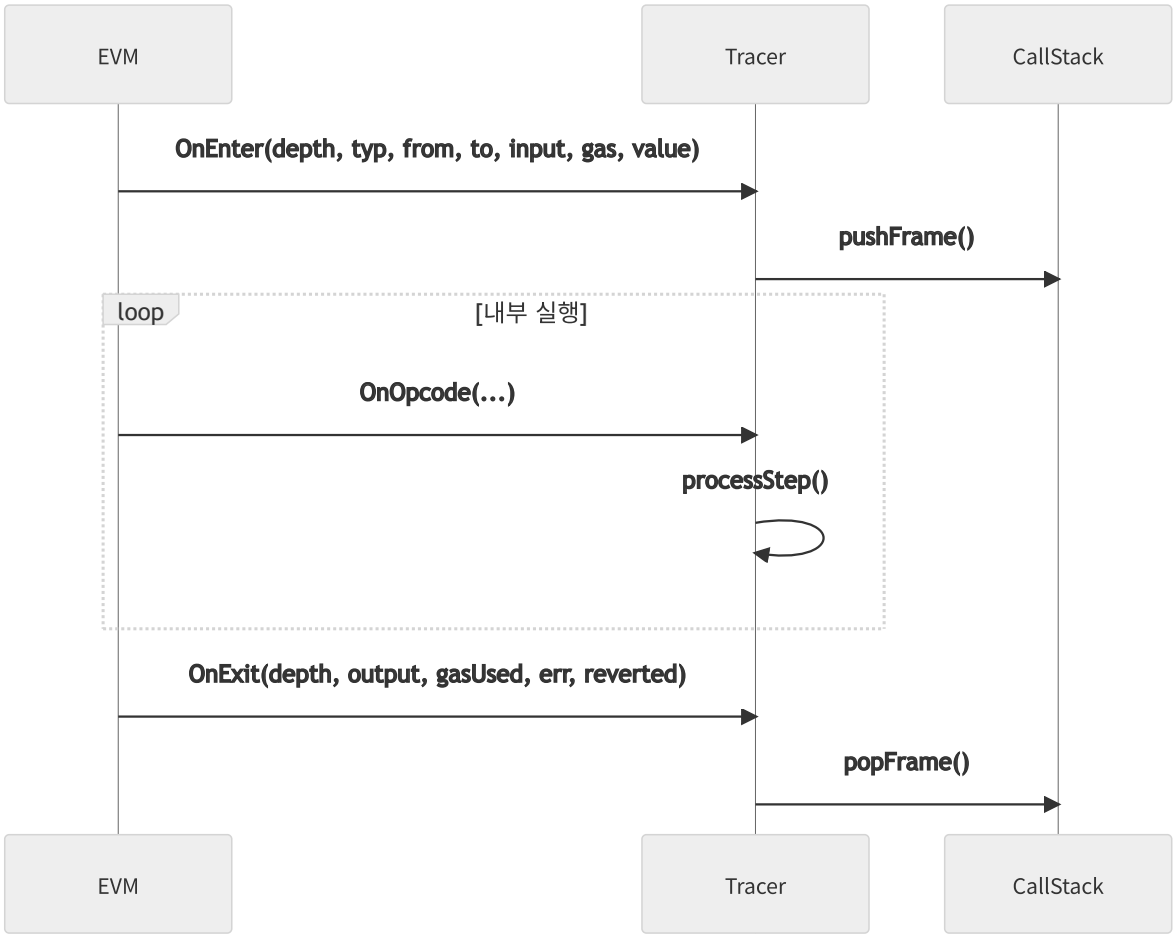
1. 트레이싱 초기화


Client
API
Tracer
EVM
TraceTransaction(hash, config)
blockByHash()
stateAtTransaction()
newTracer(code, ctx, cfg)
setupHooks()
ApplyMessage(tracer)
Client
API
Tracer
EVM

2. 트랜잭션 실행 추적


EVM
Hooks
Tracer
Output
OnTxStart(env, tx, from)
OnTxStart()
loop
[각 opcode 실행]
OnOpcode(pc, op, gas, cost, scope, rData, depth, err)
OnOpcode()
recordStep()
OnTxEnd(receipt, err)
OnTxEnd()
GetResult()
EVM
Hooks
Tracer
Output

3. 호출 프레임 추적



4. 상태 변경 추적

SLOAD/SSTORE 추적:

```
func (l *StructLogger) OnOpcode(pc  uint64, opcode  byte, gas, cost  uint64, sco

var (

op = vm.OpCode(opcode)

contractAddr = scope.Address()

stack = scope.StackData()

stackLen = len(stack)
```

```
)

// 스토리지 접근 추적

if !l.cfg.DisableStorage && (op == vm.SLOAD || op == vm.SSTORE) {

if  l.storage[contractAddr] == nil {

l.storage[contractAddr] = make(Storage)

}

if  op == vm.SLOAD && stackLen >= 1 {

address := common.Hash(stack[stackLen-1].Bytes32())

value := l.env.StateDB.GetState(contractAddr, address)

l.storage[contractAddr][address] = value

} else  if  op == vm.SSTORE && stackLen >= 2 {

value := common.Hash(stack[stackLen-2].Bytes32())

address := common.Hash(stack[stackLen-1].Bytes32())

l.storage[contractAddr][address] = value

}

}

}
```

JavaScript vs Go 트레이서 차이점

성능 비교

| 특성 | JavaScript 트레이서 | Go 트레이서 |

|------|----------|----------|

| 실행 속도 | 느림 (인터프리터) | 빠름 (네이티브) |

| 메모리 사용량 | 높음 | 낮음 |

| 유연성 | 매우 높음 | 제한적 |

| 커스터마이징 | 동적 | 정적 |

| 디버깅 | 어려움 | 쉬움 |

사용 사례별 선택 기준

JavaScript 트레이서 사용 시기:

- 복잡한 분석 로직이 필요한 경우
- 동적으로 트레이싱 로직을 변경해야 하는 경우
- 특정 패턴이나 조건을 찾는 경우
- 프로토타이핑 및 실험적 분석

Go 트레이서 사용 시기:

- 대량의 트랜잭션을 빠르게 처리해야 하는 경우
- 표준화된 로그 형식이 필요한 경우
- 프로덕션 환경에서의 지속적 모니터링
- 메모리 사용량을 최소화해야 하는 경우

병렬 처리 최적화

JavaScript 트레이서의 병렬 처리:

```
func (api *API) traceBlockParallel(ctx context.Context, block *types.Block, stat

var (
```

```
txs = block.Transactions()

results = make([]*txTraceResult, len(txs))

pend sync.WaitGroup

)

threads := runtime.NumCPU()

if threads > len(txs) {

threads = len(txs)

}

jobs := make(chan *txTraceTask, threads)

// 워커 고루틴 시작

for th := 0; th < threads; th++ {

pend.Add(1)

go func() {

defer pend.Done()

for task := range jobs {

// 각 트랜잭션을 별도 상태에서 추적

res, err := api.traceTx(ctx, txs[task.index], msg, txctx, blockCtx, task.statedb

if err != nil {

results[task.index] = &txTraceResult{TxHash: txs[task.index].Hash(), Error: err.

continue

}

results[task.index] = &txTraceResult{TxHash: txs[task.index].Hash(), Result: res

}
```

```
}()

}

// 작업 분배

for  i, tx := range  txs {

task := &txTraceTask{statedb: statedb.Copy(), index: i}

jobs <- task

// 다음 트랜잭션을 위한 상태 업데이트

msg, _ := core.TransactionToMessage(tx, signer, block.BaseFee())

statedb.SetTxContext(tx.Hash(), i)

core.ApplyMessage(evm, msg, new(core.GasPool).AddGas(msg.GasLimit()))

statedb.Finalise(evm.ChainConfig().IsEIP158(block.Number()))

}

close(jobs)

pend.Wait()

return  results, nil

}
```

성능 최적화

1. 메모리 관리

스트리밍 모드:

```
func  NewStreamingStructLogger(cfg *Config, writer io.Writer) *StructLogger {
```

```
l := NewStructLogger(cfg)

l.writer = writer  // 즉시 출력으로 메모리 사용량 최소화

return  l

}
```

결과 크기 제한:

```
func (l *StructLogger) OnOpcode(...) {

if  l.cfg.Limit != 0 && l.resultSize > l.cfg.Limit {

return  // 크기 제한 초과 시 추적 중단

}

entry := log.toLegacyJSON()

l.resultSize += len(entry)

l.logs = append(l.logs, entry)

}
```

2. 선택적 데이터 캡처

```
type Config struct {

EnableMemory  bool  // 메모리 캡처 (큰 오버헤드)

DisableStack  bool  // 스택 캡처 비활성화

DisableStorage  bool  // 스토리지 캡처 비활성화

EnableReturnData  bool  // 반환 데이터 캡처
```

```
}
```

3. 중단 메커니즘

```
func (l *StructLogger) Stop(err error) {
l.reason = err
l.interrupt.Store(true) // 원자적 중단 플래그
}

func (l *StructLogger) OnOpcode(...) {
if l.interrupt.Load() {
return // 중단 요청 시 즉시 종료
}
// 트레이싱 로직 계속
}
```

실제 사용 예시

1. 기본 트랜잭션 추적

```
# RPC 호출
curl -X POST --data '{
"jsonrpc": "2.0",
"method": "debug_traceTransaction",
```

```
"params": [
"0x...",
{
"tracer": "callTracer",
"timeout": "10s"
}
],
"id": 1
}'  http://localhost:8545
```

2. 커스텀 JavaScript 트레이서

```
{
callStack: [],
step: function(log, db) {
if (log.op.toString() == "CALL") {
this.callStack.push({
from: log.contract.getAddress(),
to: log.stack.peek(1),
value: log.stack.peek(2)
});
}
},
```

```
result: function() {
return {
calls:  this.callStack
};
},
fault: function(log, db) {
return {
error:  "execution failed",
pc: log.getPC(),
op: log.op.toString()
};
}
}
```

3. 블록 전체 추적

```
// 블록 내 모든 트랜잭션 추적
results, err := api.TraceBlockByNumber(ctx, rpc.BlockNumber(12345678), &TraceCon
Tracer: &structLogger,
Config: &logger.Config{
EnableMemory: false,
DisableStack: false,
DisableStorage: true,
```

```
Limit: 1000000,
},
})
```

4. 체인 범위 추적

```
// 구독 기반 체인 추적
subscription, err := api.TraceChain(ctx,
rpc.BlockNumber(12345678),
rpc.BlockNumber(12345688),
&TraceConfig{
Tracer: &gasTracker,
Reexec: &reexecBlocks,
})
```

관련 시스템

의존성 관계

1. **EVM 실행 엔진**: 트레이싱 훅 제공
2. **상태 관리 시스템**: 상태 변경 추적
3. **RPC API 시스템**: 외부 인터페이스 제공
4. **블록체인 동기화**: 과거 상태 재구성

상호작용 패턴

- **EVM → Tracer**: 실행 이벤트 전달
- **Tracer → StateDB**: 상태 정보 조회
- **API → Backend**: 블록/트랜잭션 데이터 조회
- **Client → API**: 트레이싱 요청 및 결과 수신

영향 범위

- **성능**: 트레이싱 활성화 시 실행 속도 저하
- **메모리**: 대량의 추적 데이터로 인한 메모리 사용량 증가
- **네트워크**: 결과 전송으로 인한 대역폭 사용
- **스토리지**: 추적 결과 저장 시 디스크 공간 사용

이 트랜잭션 트레이싱 시스템은 Ethereum 개발자들에게 강력한 디버깅과 분석 도구를 제공하며, JavaScript와 Go 트레이서의 조합을 통해 유연성과 성능을 모두 만족시키는 솔루션을 제공합니다.